

Algorithm for file updates in Python

Project description

I am developing a Python algorithm to manage network access controls by comparing an allow list of authorized IP addresses against a remove list. The algorithm identifies and removes unauthorized IP addresses from the allow list of a restricted healthcare network.

Open the file that contains the allow list

I assigned the file name `"allow_list.txt"` to the variable `import_file`:

```
# Assign `import_file` to the name of the file  
  
import_file = "allow_list.txt"
```

After that, I used a `with` statement with the `open()` function in read mode `"r"` to open the file:

```
# Open the file using `with` statement  
  
with open(import_file, "r") as file:
```

It allows me to access the IP addresses stored in the allow list. The `with` keyword manages system resources by automatically closing the file after the block is executed. In the statement `with open(import_file, "r") as file:`, the `open()` function takes two parameters: the file name to open and the mode of access. The `"r"` parameter indicates read mode. The `as file` part assigns the opened file object to the variable `file`, which is used to work with the file inside the `with` block.

Read the file contents

I used the `.read()` method to read the contents of the file opened in read mode `"r"`:

```
# Build `with` statement to read in the initial contents of the file  
  
with open(import_file, "r") as file:  
  
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`  
  
    ip_addresses = file.read()
```

This method converts the file contents into a string. I applied `.read()` to the `file` variable within the `with` statement, and stored the resulting string in the variable `ip_addresses`. It

allows the data from `"allow_list.txt"` to be used later for processing, organizing, and extracting IP address information in the Python program.

Convert the string into a list

To remove individual IP addresses from the allow list easily, I used the `.split()` method to convert the `ip_addresses` string into a list:

```
# Use `.split()` to convert `ip_addresses` from a string to a list  
ip_addresses = ip_addresses.split()
```

The `.split()` method separates a string into list elements based on whitespace (spaces between words or data) by default. I applied `.split()` to the `ip_addresses` variable and reassigned the result back to `ip_addresses`, creating a list of IP addresses that can be processed in later steps of the algorithm.

Iterate through the remove list

I used a `for` loop to iterate through the IP addresses in the `remove_list`:

```
# Build iterative statement  
# Name loop variable `element`  
# Loop through `ip_addresses`  
for element in ip_addresses:
```

The `for` loop allows code to be repeated for each element in a sequence. In this algorithm, the loop assigns each IP address in the list to the loop variable `element` using the `in` keyword, which specifies the sequence to iterate through. It allows the program to process each IP address individually for removal from the allow list.

Remove IP addresses that are on the remove list

My algorithm removes any IP addresses from `ip_addresses` that are also found in `remove_list`:

```

for element in remove_list:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in ip_addresses:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

```

Inside a `for` loop, I used a conditional statement to check whether each `element` from `remove_list` exists in `ip_addresses`. The check prevents errors that would occur if `.remove()` is applied to a value not in the list. If the element is found, I use the `.remove()` method with `element` as the argument to remove it from `ip_addresses`, ensuring that all unauthorized IP addresses are removed from the allow list.

Update the file with the revised list of IP addresses

To update the allow list file with the revised IP addresses, I first used the `.join()` method to convert the `ip_addresses` list back into a string:

```

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = "\n".join(ip_addresses)

```

I used `"\n"` as the separator so each IP address appears on a new line in the file. Then, I used a `with` statement with the `open()` function in write mode `"w"` to open `"allow_list.txt"`. This mode overwrites the existing file content. Inside the `with` block, I used the `.write()` method to write the updated string back to the file, ensuring that removed IP addresses no longer have access to restricted content.

Summary

I created an algorithm that removes IP addresses in the `remove_list` from the `"allow_list.txt"` file of approved IP addresses. The process began by opening the file, reading its contents as a string, and converting that string into a list stored in `ip_addresses`. I then iterated through each IP address in `remove_list` and checked whether it existed in `ip_addresses`. If a match was found, I used the `.remove()` method to delete it from the list. Finally, I used the `.join()` method to convert the updated list back into a string and wrote it back to the file, overwriting the original contents with the updated allow list.